

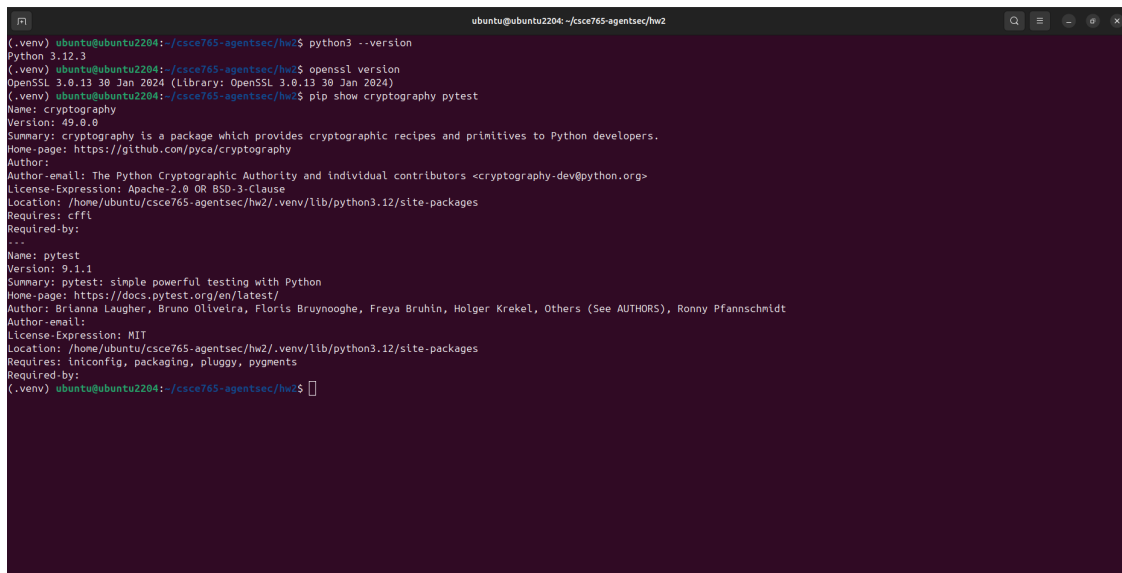
Homework 2: Protect Agent Messages with Classic Cryptography

Amesh Shrestha
CSCE 465/765 — Computer & Network Security

October 4, 2026

Lab Preparation

The work was done inside the course VM using NAT networking. Python 3.12.3 and OpenSSL 3.0.13 were used. The virtual environment was located at `/home/ubuntu/csce765-agentsec/hw2/.venv`, with `cryptography=49.0.0` and `pytest=9.1.1` installed.

A terminal window with a dark purple background and white text. The window title is 'ubuntu@ubuntu2204: ~/csce765-agentsec/hw2'. The terminal shows the following commands and output:

```
(.venv) ubuntu@ubuntu2204:~/csce765-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2204:~/csce765-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2204:~/csce765-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce765-agentsec/hw2/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
...
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce765-agentsec/hw2/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2204:~/csce765-agentsec/hw2$
```

Figure 1: Task 0 environment and package versions.

The Diffie–Hellman parameter file `ffdhe3072.pem` was also generated and checked with OpenSSL. The saved result reported a 3072-bit `ffdhe3072` group.

1 Task 1: Why Encryption Alone Is Insufficient

1.1 Design and Results

baseline_ctr.py was written using AES-CTR without a MAC. The assigned plaintext was

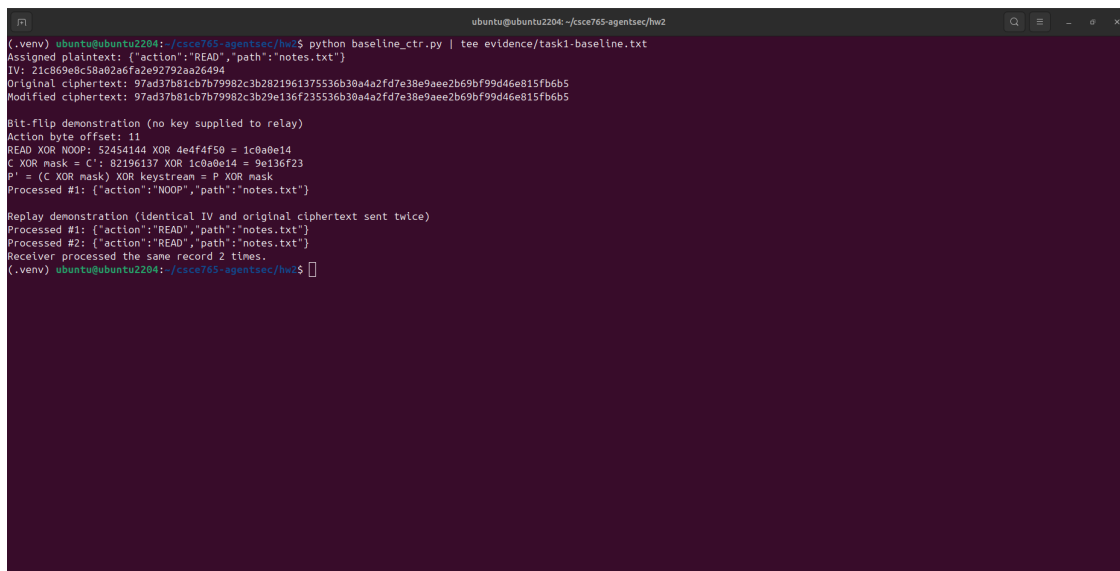
$\{\text{"action": "READ", "path": "notes.txt"}\}$

A relay was given only the ciphertext and changed the four bytes for READ into NOOP by applying the XOR difference between the two strings.

The saved run showed:

```
READ XOR NOOP: 52454144 XOR 4e4f4f50 = 1c0a0e14
C XOR mask = C': 82196137 XOR 1c0a0e14 = 9e136f23
Processed #1: {"action": "NOOP", "path": "notes.txt"}
```

The original ciphertext was also replayed twice. Both copies were accepted and the same READ command was processed two times.



```
ubuntu@ubuntu2204: ~/csc765-agentsec/hw2
(.venv) ubuntu@ubuntu2204: ~/csc765-agentsec/hw2$ python baseline_ctr.py | tee evidence/task1-baseline.txt
Assigned plaintext: {"action": "READ", "path": "notes.txt"}
IV: 21c869e8c5a02a6fa2e92792aa26494
Original ciphertext: 97ad37b81cb7b79982c3b2821961375536b30a4a2fd7e38e9aee2b69bf99d46e815fb6b5
Modified ciphertext: 97ad37b81cb7b79982c3b29e136f235536b30a4a2fd7e38e9aee2b69bf99d46e815fb6b5

Bit-flip demonstration (no key supplied to relay)
Action byte offset: 11
READ XOR NOOP: 52454144 XOR 4e4f4f50 = 1c0a0e14
C XOR mask = C': 82196137 XOR 1c0a0e14 = 9e136f23
P' = (C XOR mask) XOR keystream = P XOR mask
Processed #1: {"action": "NOOP", "path": "notes.txt"}

Replay demonstration (identical IV and original ciphertext sent twice)
Processed #1: {"action": "READ", "path": "notes.txt"}
Processed #2: {"action": "READ", "path": "notes.txt"}
Receiver processed the same record 2 times.
(.venv) ubuntu@ubuntu2204: ~/csc765-agentsec/hw2$
```

Figure 2: Task 1 bit-flip and replay demonstration.

1.2 Root Cause

CTR encryption uses $C = P \oplus S$. If a known XOR mask is applied to the ciphertext, the same mask appears in the decrypted plaintext. Since no MAC was used, the change was not detected.

Replay was possible because no freshness check or accepted-sequence state was present. The same valid IV and ciphertext could therefore be processed again.

2 Task 2: Authenticated Diffie–Hellman Setup

2.1 Design

`handshake.py` was used to simulate a gateway and node in one process. Each party had its own identity, role, long-term 3072-bit RSA signing key, expected peer identity, and trusted peer RSA public key.

Fresh FFDHE3072 private values and fresh 16-byte nonces were generated for every session. The canonical transcript contained the protocol label, group identifier, both identities, both DH public values, and both nonces. Every field was length-prefixed with a 4-byte big-endian length, and malformed lengths were rejected before hashing.

The gateway signed

`gateway || SHA256(transcript),`

and the node signed the same transcript hash with the `node` role prefix. Peer identity and signature checks were completed before session keys were released.

The assignment KDF was used exactly as specified:

```
K_master = SHA-256("CSCE465-KDF-v1" || Z || TH)

K_g2n_enc = HMAC-SHA-256(K_master,
                           "gateway-to-node encryption" || TH)
K_g2n_mac = HMAC-SHA-256(K_master,
                           "gateway-to-node MAC" || TH)
K_n2g_enc = HMAC-SHA-256(K_master,
                           "node-to-gateway encryption" || TH)
K_n2g_mac = HMAC-SHA-256(K_master,
                           "node-to-gateway MAC" || TH)

session_id = first 8 bytes of HMAC-SHA-256(
    K_master, "session identifier" || TH)
```

2.2 Results and Security

Both parties produced matching session keys. Four distinct 32-byte keys and one 8-byte session ID were produced, and a second session produced a different session ID.

```
ubuntu@ubuntu2204: ~/csc765-agentsec/hw2
(.venv) ubuntu@ubuntu2204:~/csc765-agentsec/hw2$ python handshake.py | tee evidence/task2-handshake.txt
Handshake passed; both parties derived matching keys.
Four distinct 32-byte keys; 8-byte session ID: e9c87870e363c215
Second session passed with a new session ID: aec6ad4fdbdc4cf8f
(.venv) ubuntu@ubuntu2204:~/csc765-agentsec/hw2$
```

Figure 3: Task 2 successful handshake and fresh second session.

The RSA signatures were used to authenticate the ephemeral DH exchange. Because the identities, roles, nonces, and DH values were bound to the signed transcript, basic substitution and reflection attempts were rejected. Fresh nonces and DH values also prevented an old signed exchange from matching a new session.

Ephemeral DH gives forward secrecy against later compromise of only the long-term RSA signing key, assuming the old ephemeral secrets are no longer available. Separate KDF labels were used for encryption, MAC, and traffic direction so those keys were not reused for different purposes.

Without the signatures, DH would not authenticate the peer. An on-path attacker could replace the DH public values, establish separate secrets with each endpoint, and relay or modify later traffic.

3 Task 3: Encrypt-then-MAC Record Layer

3.1 Design and Results

`secure_record.py` was kept separate from the handshake code. Each endpoint used the keys and session ID returned by Task 2 and maintained separate send and receive sequence counters.

The record format was:

$$\text{header} \parallel \text{ciphertext} \parallel \text{tag}$$

with

$$\text{IV} = \text{session_id} \parallel \text{sequence},$$

$$\text{ciphertext} = \text{AES-256-CTR}(K_{\text{enc}}, \text{IV}, \text{plaintext}),$$

and

$$\text{tag} = \text{HMAC-SHA256}(K_{\text{mac}}, \text{header} \parallel \text{IV} \parallel \text{ciphertext}).$$

The receiver checked record length and HMAC before decryption, then checked the version, direction, message type, and exact expected sequence number. On failure, no plaintext was returned and the receive counter was not advanced.

The normal run showed gateway-to-node sequences 0 and 1, while the node-to-gateway direction independently started at sequence 0.

```

(.venv) ubuntu@ubuntu2204:~/csc765-agentsec/hw2$ python secure_record.py | tee evidence/task3-records.txt
gateway -> node, sequence 0: {"action": "READ", "path": "notes.txt"}
node -> gateway, sequence 0: OK
gateway -> node, sequence 1: {"action": "READ", "path": "notes.txt"}
Bidirectional messages and sequence checks passed.
(.venv) ubuntu@ubuntu2204:~/csc765-agentsec/hw2$

```

Figure 4: Task 3 bidirectional record exchange.

3.2 Discovered CTR Counter-Overlap Vulnerability

A confidentiality problem was found in the required IV layout. Each record starts with a different IV, but CTR increments its counter for every 16-byte block. This creates the following overlap:

Record	Block	Counter input
Sequence 0	0	<code>session_id 0</code>
Sequence 0	1	<code>session_id 1</code>
Sequence 1	0	<code>session_id 1</code>

The second block of sequence 0 and the first block of sequence 1 therefore use the same CTR keystream. In the saved experiment, 32 bytes of A were sent first and 16 bytes of B were sent next. The ciphertext XOR matched the plaintext XOR, and the second plaintext was recovered from the known first plaintext.

```

ubuntu@ubuntu2204: ~/csc765-agentsec/hw2
(.venv) ubuntu@ubuntu2204: ~/csc765-agentsec/hw2 $ python -c 'PV' | tee evidence/task3-ctr-overlap.txt
from handshake import GATEWAY, NODE, create_demo_parties, run_handshake
from secure_record import Endpoint, HEADER, HEADER_SIZE, TAG_SIZE

gateway_keys, node_keys = run_handshake(*create_demo_parties())
gateway = Endpoint(gateway_keys, GATEWAY)
node = Endpoint(node_keys, NODE)

plaintext0 = b'A' * 32
plaintext1 = b'B' * 16
record0 = gateway.seal(plaintext0)
record1 = gateway.seal(plaintext1)

assert node.open_record(record0) == plaintext0
assert node.open_record(record1) == plaintext1
print("Both records passed authentication and were accepted.")
print("Record sequences:",
      HEADER.unpack(record0[HEADER_SIZE:][2]),
      HEADER.unpack(record1[HEADER_SIZE:][2]))

ciphertext0 = record0[HEADER_SIZE:TAG_SIZE]
ciphertext1 = record1[HEADER_SIZE:TAG_SIZE]

block0 = ciphertext0[16:32]
block1 = ciphertext1[:16]
ciphertext_xor = bytes(a ^ b for a, b in zip(block0, block1))
plaintext_xor = bytes(a ^ b for a, b in zip(plaintext0[16:32], plaintext1))

print("Record 0, ciphertext block 1:", block0.hex())
print("Record 1, ciphertext block 0:", block1.hex())
print("Ciphertext blocks XOR:", ciphertext_xor.hex())
print("Plaintext blocks XOR:", plaintext_xor.hex())
assert ciphertext_xor == plaintext_xor
print("MATCH: the two blocks reused the same CTR keystream.")

recovered = bytes(c ^ known for c, known
                  in zip(ciphertext_xor, plaintext0[16:32]))
print("Recovered second plaintext:", recovered)
assert recovered == plaintext1
print("Recovery succeeded using ciphertexts and the known first block.")
PV
Both records passed authentication and were accepted.
Record sequences: 0 1
Record 0, ciphertext block 1: 61d3f08eaa30508caac0b05e6ae5b1f
Record 1, ciphertext block 0: 62083c03c0a08080c9f0b0e5e6501c
Ciphertext blocks XOR: 03030303030303030303030303030303
Plaintext blocks XOR: 03030303030303030303030303030303
MATCH: the two blocks reused the same CTR keystream.
Recovered second plaintext: 01888080808080808080808080808080
Recovery succeeded using ciphertexts and the known first block.
(.venv) ubuntu@ubuntu2204: ~/csc765-agentsec/hw2 $

```

Figure 5: Task 3 CTR counter-overlap experiment and plaintext recovery.

The HMAC did not prevent this because neither valid ciphertext was changed. The problem was caused by keystream reuse, not an integrity failure.

One possible fix would be to limit each record to at most 16 plaintext bytes and fragment longer messages into separately authenticated records. This was not implemented.

4 Task 4: Adversarial Tests

The permanent test suite was run with:

```
python -m pytest -v tests/ 2>&1 | tee evidence/task4-tests.txt
```

The saved result was:

```
===== 43 passed in 1.62s =====
```

There were 24 handshake cases and 19 record-layer cases. The required attack cases all produced safe failures:

Required case	Observed result
Valid handshake and bidirectional messages	Matching session keys and correct plaintext were accepted.
Modified ciphertext	Rejected with MAC .
Modified authenticated header	Rejected with MAC ; inconsistent length was rejected with length .
Replayed record	First copy was accepted; replay was rejected with sequence .
Record reflected into opposite direction	Rejected with MAC .
Wrong RSA key, bad signature, or reflected handshake	Rejected with signature or identity , depending on the case.

Additional tests covered malformed transcripts, changed nonces and DH values, replayed handshake messages, exact KDF behavior, wrong session records, changed tags, truncation, unsupported headers, empty plaintext, and sequence exhaustion. Rejected records were also checked to make sure decryption was not reached and counters were unchanged.

5 Security Note

The attacker is assumed to sit on the path between the gateway and node. Messages can be observed, changed, replayed, reordered, reflected, or dropped. The attacker is not assumed to have either endpoint's private keys or control of the endpoint process. Each party is assumed to already have the correct RSA public key for the other party. The protocol is not meant to establish that initial trust, protect a compromised endpoint, hide message length or timing, or guarantee availability. Communication can still be stopped by dropping messages.

CTR mode requires that the same counter input never be reused with the same encryption key. Reusing the key for many blocks is normal, but reusing a key and counter repeats the same keystream. If two ciphertext blocks use that same keystream, XORing them gives the XOR of the two plaintext blocks. If one plaintext is known, the other can then be recovered. The AES key itself is not recovered.

A limitation was found in the IV format required by the assignment. The IV is built as `session_id || sequence`, so each record starts with a different IV. However, CTR increments the counter once for every 16-byte block. This means block 1 of sequence 0 uses the same counter value as block 0 of sequence 1. The saved Task 3 experiment showed this directly: the ciphertext XOR matched the plaintext XOR, and the second plaintext was recovered from known plaintext. Both records still passed HMAC because neither record was modified. A simple possible fix would be to limit each record to 16 plaintext bytes and split longer messages into separately authenticated records. That fix was not implemented because the assigned construction was kept and the issue was only documented, following the professor's direction.

Verification is done before decryption so unauthenticated ciphertext is never released as application input. Record length and HMAC are checked first, followed by the expected version, direction, type, and sequence number. If a check fails, no plaintext is returned and the receive counter is not advanced. Replay and out-of-order records are also rejected even when their HMAC is otherwise valid. HMAC protects integrity, but it does not repair confidentiality that has already been lost because of CTR keystream reuse.

Separate encryption and MAC keys are derived because the two keys have different purposes. Different KDF labels are also used for gateway-to-node and node-to-gateway traffic. This keeps the directions and cryptographic operations separated. It also helps reflected records fail because the opposite direction uses different authentication material. Key separation does not fix a bad counter layout, so the CTR overlap still has to be treated as a separate issue.

AES-GCM would combine encryption and authentication into one AEAD operation. The header could be supplied as associated data so it would be authenticated without being encrypted. That would reduce the amount of manual Encrypt-then-MAC construction used here. GCM still requires unique nonces under a key, so a correct nonce design and sequence handling would still be required.

Finally, a valid protected record only proves that the message was produced by someone with the session key and that the protected bytes were not changed in transit. It does not prove that the requested action is safe. A compromised or misled authorized agent could still create a valid request to delete important files or perform another dangerous action. Permission checks, restricted tool capabilities, and argument validation are still required at the application layer.